

Exercícios Práticos de Encapsulamento

Parte 1: Exercícios em Java (Foco em Rigidez e Controle)

Exercício 1: Classe `Produto` (Nível Básico)

O objetivo é criar uma classe para representar um produto, aplicando o encapsulamento básico com atributos privados e métodos de acesso.

Requisitos:

1. Crie uma classe chamada `Produto`.
2. Adicione os seguintes atributos privados:
 - `String nome`
 - `double preco`
 - `int quantidadeEmEstoque`
3. Crie um construtor que receba o nome e o preço para inicializar o objeto. A quantidade em estoque deve ser inicializada como 0.
4. Implemente métodos `get` (getters) para todos os atributos (`getNome()`, `getPreco()`, `getQuantidadeEmEstoque()`).[1]
5. Implemente um método `set` (setter) apenas para o `preco` (`setPreco(double novoPreco)`). Este método deve incluir uma validação: o preço não pode ser negativo. Se um valor negativo for fornecido, o preço não deve ser alterado.[1, 2]
6. Crie métodos públicos para a lógica de negócio:
 - `adicionarEstoque(int quantidade)`: Aumenta a quantidade em estoque.
 - `removerEstoque(int quantidade)`: Diminui a quantidade em estoque, validando se há estoque suficiente.

Exercício 2: Classe `Carro` (Nível Intermediário)

Este exercício foca em encapsular o comportamento e o estado de um objeto, garantindo que as transições de estado sejam sempre válidas.

Requisitos:

1. Crie uma classe chamada `Carro`.
2. Adicione os seguintes atributos privados:
 - `String modelo`
 - `int ano`
 - `int velocidadeAtual`
3. Crie um construtor que inicialize o `modelo` e o `ano`. A `velocidadeAtual` deve começar em 0.
4. Implemente um método `get` para cada atributo.
5. Não crie um método `set` para `velocidadeAtual`. A velocidade só deve ser alterada através dos métodos de comportamento.
6. Crie os seguintes métodos públicos que encapsulam o comportamento do carro:
 - `acelerar(int incremento)`: Aumenta a `velocidadeAtual` em `incremento`, mas não pode ultrapassar uma velocidade máxima de 200.
 - `frear(int decremento)`: Diminui a `velocidadeAtual` em `decremento`, mas não pode resultar em uma velocidade negativa (o mínimo é 0).

Parte 2: Exercícios em Python (Foco em Convenção e Propriedades)

Exercício 1: Classe `Aluno` (Nível Básico)

O objetivo é praticar o uso das convenções de nomenclatura com underscores para sinalizar a intenção de uso dos atributos.

Requisitos:

1. Crie uma classe chamada `Aluno`.
2. No construtor (`\_\_init\_\_`), defina os seguintes atributos:
 - `self.\_nome` (atributo "protegido" para uso interno e por subclasses).[3, 4]
 - `self.\_\_matricula` (atributo "privado" para evitar conflitos de nome).[4, 5]
3. Crie um método público `get\_matricula()` que retorne o valor de `\_\_matricula`.
4. Tente acessar `aluno.\_nome` e `aluno.\_\_matricula` diretamente de uma instância da classe para observar o que acontece (especialmente com o atributo de duplo underscore).

Exercício 2: Classe `Retangulo` com `@property` (Nível Intermediário)

Este exercício foca na forma "pythônica" de implementar getters e setters, usando o decorador `@property` para uma API limpa e com validação.

Requisitos:

1. Crie uma classe chamada `Retangulo`.
2. No construtor, inicialize os atributos internos `\_largura` e `\_altura`.
3. Use o decorador `@property` para criar uma propriedade `largura` [6, 7, 8]:
 - O getter deve retornar o valor de `\_largura`.
 - O setter (`@largura.setter`) deve validar se o novo valor é positivo antes de atribuí-lo a `\_largura`. Caso contrário, imprima uma mensagem de erro.
4. Faça o mesmo para a propriedade `altura`.
5. Crie uma propriedade **somente leitura** chamada `area`:
 - Use `@property` para definir um método `area` que calcula e retorna o resultado de `self.largura \* self.altura`.
 - Não implemente um setter para `area`, tornando-a um atributo calculado e não modificável diretamente.